

```

float LoadSingle(Platform::String^ key, float defaultValue);

DirectX::XMFLOAT3 LoadXMFLOAT3(
    Platform::String^ key,
    DirectX::XMFLOAT3 defaultValue);

Platform::String^ LoadString(
    Platform::String^ key,
    Platform::String^ defaultValue);

private:
    Platform::String^ m_keyName;
    Windows::Foundation::Collections::IPropertySet^ m_settingsValues;
};

```

The **MarbleMazeMain** class holds a **PersistentState** object. The **MarbleMazeMain** constructor initializes this object and provides the local application data store as the backing data store.

```

C++

m_persistentState = ref new PersistentState();

m_persistentState->Initialize(
    Windows::Storage::ApplicationData::Current->LocalSettings->Values,
    "MarbleMaze");

```

Marble Maze saves its state when the marble passes over a checkpoint or the goal (in the **MarbleMazeMain::Update** method), and when the window loses focus (in the **MarbleMazeMain::OnFocusChange** method). If your game holds a large amount of state data, we recommend that you occasionally save state to persistent storage in a similar manner because you only have a few seconds to respond to the suspend notification. Therefore, when your app receives a suspend notification, it only has to save the state data that has changed.

To respond to suspend and resume notifications, the **MarbleMazeMain** class defines the **SaveState** and **LoadState** methods that are called on suspend and resume. The **MarbleMazeMain::OnSuspending** method handles the suspend event and the **MarbleMazeMain::OnResuming** method handles the resume event.

The **MarbleMazeMain::OnSuspending** method saves game state and suspends audio.

```

C++

void MarbleMazeMain::OnSuspending()
{
    SaveState();
}

```

```
m_audio.SuspendAudio();  
}
```

The **MarbleMazeMain::SaveState** method saves game state values such as the current position and velocity of the marble, the most recent checkpoint, and the high-score table.

C++

```
void MarbleMazeMain::SaveState()  
{  
    m_persistentState->SaveXMFLOAT3(":Position", m_physics.GetPosition());  
    m_persistentState->SaveXMFLOAT3(":Velocity", m_physics.GetVelocity());  
  
    m_persistentState->SaveSingle(  
        ":ElapsedTime",  
        m_inGameStopwatchTimer.GetElapsedTime());  
  
    m_persistentState->SaveInt32(":GameState", static_cast<int>  
(m_gameState));  
    m_persistentState->SaveInt32(":Checkpoint", static_cast<int>  
(m_currentCheckpoint));  
  
    int i = 0;  
    HighScoreEntries entries = m_highScoreTable.GetEntries();  
    const int bufferLength = 16;  
    char16 str[bufferLength];  
  
    m_persistentState->SaveInt32(":ScoreCount", static_cast<int>  
(entries.size()));  
  
    for (auto iter = entries.begin(); iter != entries.end(); ++iter)  
    {  
        int len = swprintf_s(str, bufferLength, L"%d", i++);  
        Platform::String^ string = ref new Platform::String(str, len);  
  
        m_persistentState->SaveSingle(  
            Platform::String::Concat(":ScoreTime", string),  
            iter->elapsedTime);  
  
        m_persistentState->SaveString(  
            Platform::String::Concat(":ScoreTag", string),  
            iter->tag);  
    }  
}
```

When the game resumes, it only has to resume audio. It doesn't have to load state from persistent storage because the state is already loaded in memory.

How the game suspends and resumes audio is explained in the document [Adding audio to the Marble Maze sample](#).

To support restart, the **MarbleMazeMain** constructor, which is called during startup, calls the **MarbleMazeMain::LoadState** method. The **MarbleMazeMain::LoadState** method reads and applies the state to the game objects. This method also sets the current game state to paused if the game was paused or active when it was suspended. We pause the game so that the user is not surprised by unexpected activity. It also moves to the main menu if the game was not in a gameplay state when it was suspended.

C++

```
void MarbleMazeMain::LoadState()
{
    XMFLOAT3 position = m_persistentState->LoadXMFLOAT3(
        ":Position",
        m_physics.GetPosition());

    XMFLOAT3 velocity = m_persistentState->LoadXMFLOAT3(
        ":Velocity",
        m_physics.GetVelocity());

    float elapsedTime = m_persistentState->LoadSingle(":ElapsedTime", 0.0f);

    int gameState = m_persistentState->LoadInt32(
        ":GameState",
        static_cast<int>(m_gameState));

    int currentCheckpoint = m_persistentState->LoadInt32(
        ":Checkpoint",
        static_cast<int>(m_currentCheckpoint));

    switch (static_cast<GameState>(gameState))
    {
    case GameState::Initial:
        break;

    case GameState::MainMenu:
    case GameState::HighScoreDisplay:
    case GameState::PreGameCountdown:
    case GameState::PostGameResults:
        SetGameState(GameState::MainMenu);
        break;

    case GameState::InGameActive:
    case GameState::InGamePaused:
        m_inGameStopwatchTimer.SetVisible(true);
        m_inGameStopwatchTimer.SetElapsedTime(elapsedTime);
        m_physics.SetPosition(position);
        m_physics.SetVelocity(velocity);
```

```

        m_currentCheckpoint = currentCheckpoint;
        SetGameState(GameState::InGamePaused);
        break;
    }

    int count = m_persistentState->LoadInt32(":ScoreCount", 0);

    const int bufferLength = 16;
    char16 str[bufferLength];

    for (int i = 0; i < count; i++)
    {
        HighScoreEntry entry;
        int len = swprintf_s(str, bufferLength, L"%d", i);
        Platform::String^ string = ref new Platform::String(str, len);

        entry.elapsedTime = m_persistentState->LoadSingle(
            Platform::String::Concat(":ScoreTime", string),
            0.0f);

        entry.tag = m_persistentState->LoadString(
            Platform::String::Concat(":ScoreTag", string),
            L "");

        m_highScoreTable.AddScoreToTable(entry);
    }
}

```

Important

Marble Maze doesn't distinguish between cold starting—that is, starting for the first time without a prior suspend event—and resuming from a suspended state. This is recommended design for all UWP apps.

For more info about application data, see [Store and retrieve settings and other app data](#).

Next steps

Read [Adding visual content to the Marble Maze sample](#) for information about some of the key practices to keep in mind when you work with visual resources.

Related topics

- [Adding visual content to the Marble Maze sample](#)
- [Marble Maze sample fundamentals](#)
- [Developing Marble Maze, a UWP game in C++ and DirectX](#)

Adding visual content to the Marble Maze sample

Article • 10/27/2022

This document describes how the Marble Maze game uses Direct3D and Direct2D in the Universal Windows Platform (UWP) app environment so that you can learn the patterns and adapt them when you work with your own game content. To learn how visual game components fit in the overall application structure of Marble Maze, see [Marble Maze application structure](#).

We followed these basic steps as we developed the visual aspects of Marble Maze:

1. Create a basic framework that initializes the Direct3D and Direct2D environments.
2. Use image and model editing programs to design the 2D and 3D assets that appear in the game.
3. Ensure that 2D and 3D assets properly load and appear in the game.
4. Integrate vertex and pixel shaders that enhance the visual quality of the game assets.
5. Integrate game logic, such as animation and user input.

We also focused first on adding 3D assets and then on 2D assets. For example, we focused on core game logic before we added the menu system and timer.

We also needed to iterate through some of these steps multiple times during the development process. For example, as we made changes to the mesh and marble models, we had to also change some of the shader code that supports those models.

ⓘ Note

The sample code that corresponds to this document is found in the [DirectX Marble Maze game sample](#).

Here are some of the key points that this document discusses for when you work with DirectX and visual game content, namely, when you initialize the DirectX graphics libraries, load scene resources, and update and render the scene:

- Adding game content typically involves many steps. These steps also often require iteration. Game developers often focus first on adding 3D game content and then on adding 2D content.

- Reach more customers and give them all a great experience by supporting the greatest range of graphics hardware as possible.
- Cleanly separate design-time and run-time formats. Structure your design-time assets to maximize flexibility and enable rapid iterations on content. Format and compress your assets to load and render as efficiently as possible at run time.
- You create the Direct3D and Direct2D devices in a UWP app much like you do in a classic Windows desktop app. One important difference is how the swap chain is associated with the output window.
- When you design your game, ensure that the mesh format that you choose supports your key scenarios. For example, if your game requires collision, make sure that you can obtain collision data from your meshes.
- Separate game logic from rendering logic by first updating all scene objects before you render them.
- You typically draw your 3D scene objects, and then any 2D objects that appear in front of the scene.
- Synchronize drawing to the vertical blank to ensure that your game does not spend time drawing frames that will never be actually shown on the display. A *vertical blank* is the time between when one frame finishes drawing to the monitor and the next frame begins.

Getting started with DirectX graphics

When we planned the Marble Maze Universal Windows Platform (UWP) game, we chose C++ and Direct3D 11.1 because they are excellent choices for creating 3D games that require maximum control over rendering and high performance. DirectX 11.1 supports hardware from DirectX 9 to DirectX 11, and therefore can help you reach more customers more efficiently because you don't have to rewrite code for each of the earlier DirectX versions.

Marble Maze uses Direct3D 11.1 to render the 3D game assets, namely the marble and the maze. Marble Maze also uses Direct2D, DirectWrite, and Windows Imaging Component (WIC) to draw the 2D game assets, such as the menus and the timer.

Game development requires planning. If you are new to DirectX graphics, we recommend that you read [DirectX: Getting started](#) to familiarize yourself with the basic concepts of creating a UWP DirectX game. As you read this document and work through the Marble Maze source code, you can refer to the following resources for more in-depth information about DirectX graphics:

- [Direct3D 11 Graphics](#): Describes Direct3D 11, a powerful, hardware-accelerated 3D graphics API for rendering 3D geometry on the Windows platform.

- [Direct2D](#): Describes Direct2D, a hardware-accelerated, 2D graphics API that provides high performance and high-quality rendering for 2D geometry, bitmaps, and text.
- [DirectWrite](#): Describes DirectWrite, which supports high-quality text rendering.
- [Windows Imaging Component](#): Describes WIC, an extensible platform that provides low-level API for digital images.

Feature levels

Direct3D 11 introduces a paradigm named *feature levels*. A feature level is a well-defined set of GPU functionality. Use feature levels to target your game to run on earlier versions of Direct3D hardware. Marble Maze supports feature level 9.1 because it requires no advanced features from the higher levels. We recommend that you support the greatest range of hardware possible and scale your game content so that your customers that have either high or low-end computers all have a great experience. For more information about feature levels, see [Direct3D 11 on Downlevel Hardware](#).

Initializing Direct3D and Direct2D

A device represents the display adapter. You create the Direct3D and Direct2D devices in a UWP app much like you do in a classic Windows desktop app. The main difference is how you connect the Direct3D swap chain to the windowing system.

The **DeviceResources** class is a foundation for managing Direct3D and Direct2D. This class handles general infrastructure, not game-specific assets. Marble Maze defines the **MarbleMazeMain** class to handle game-specific assets, which has a reference to a **DeviceResources** object to give it access to Direct3D and Direct2D.

During initialization, the **DeviceResources** constructor creates device-independent resources and the Direct3D and Direct2D devices.

C++

```
// Initialize the Direct3D resources required to run.
DX::DeviceResources::DeviceResources() :
    m_screenViewport(),
    m_d3dFeatureLevel(D3D_FEATURE_LEVEL_9_1),
    m_d3dRenderTargetSize(),
    m_outputSize(),
    m_logicalSize(),
    m_nativeOrientation(DisplayOrientations::None),
    m_currentOrientation(DisplayOrientations::None),
    m_dpi(-1.0f),
    m_deviceNotify(nullptr)
```

```
{
    CreateDeviceIndependentResources();
    CreateDeviceResources();
}
```

The **DeviceResources** class separates this functionality so that it can more easily respond when the environment changes. For example, it calls the **CreateWindowSizeDependentResources** method when the window size changes.

Initializing the Direct2D, DirectWrite, and WIC factories

The **DeviceResources::CreateDeviceIndependentResources** method creates the factories for Direct2D, DirectWrite, and WIC. In DirectX graphics, factories are the starting points for creating graphics resources. Marble Maze specifies **D2D1_FACTORY_TYPE_SINGLE_THREADED** because it performs all drawing on the main thread.

C++

```
// These are the resources required independent of hardware.
void DX::DeviceResources::CreateDeviceIndependentResources()
{
    // Initialize Direct2D resources.
    D2D1_FACTORY_OPTIONS options;
    ZeroMemory(&options, sizeof(D2D1_FACTORY_OPTIONS));

    #if defined(_DEBUG)
        // If the project is in a debug build, enable Direct2D debugging via SDK
        Layers.
        options.debugLevel = D2D1_DEBUG_LEVEL_INFORMATION;
    #endif

    // Initialize the Direct2D Factory.
    DX::ThrowIfFailed(
        D2D1CreateFactory(
            D2D1_FACTORY_TYPE_SINGLE_THREADED,
            __uuidof(ID2D1Factory2),
            &options,
            &m_d2dFactory
        )
    );

    // Initialize the DirectWrite Factory.
    DX::ThrowIfFailed(
        DWriteCreateFactory(
            DWRITE_FACTORY_TYPE_SHARED,
            __uuidof(IDWriteFactory2),
            &m_dwriteFactory
        )
    );
};
```

```

// Initialize the Windows Imaging Component (WIC) Factory.
DX::ThrowIfFailed(
    CoCreateInstance(
        CLSID_WICImagingFactory2,
        nullptr,
        CLSCTX_INPROC_SERVER,
        IID_PPV_ARGS(&m_wicFactory)
    )
);
}

```

Creating the Direct3D and Direct2D devices

The **DeviceResources::CreateDeviceResources** method calls [D3D11CreateDevice](#) to create the device object that represents the Direct3D display adapter. Because Marble Maze supports feature level 9.1 and above, the

DeviceResources::CreateDeviceResources method specifies levels 9.1 through 11.1 in the **featureLevels** array. Direct3D walks the list in order and gives the app the first feature level that is available. Therefore the **D3D_FEATURE_LEVEL** array entries are listed from highest to lowest so that the app will get the highest feature level available. The **DeviceResources::CreateDeviceResources** method obtains the Direct3D 11.1 device by querying the Direct3D 11 device that's returned from **D3D11CreateDevice**.

C++

```

// This flag adds support for surfaces with a different color channel
// ordering
// than the API default. It is required for compatibility with Direct2D.
UINT creationFlags = D3D11_CREATE_DEVICE_BGRA_SUPPORT;

#ifdef _DEBUG
    if (DX::SdkLayersAvailable())
    {
        // If the project is in a debug build, enable debugging via SDK
        // Layers
        // with this flag.
        creationFlags |= D3D11_CREATE_DEVICE_DEBUG;
    }
#endif

// This array defines the set of DirectX hardware feature levels this app
// will support.
// Note the ordering should be preserved.
// Don't forget to declare your application's minimum required feature level
// in its
// description. All applications are assumed to support 9.1 unless
// otherwise stated.
D3D_FEATURE_LEVEL featureLevels[] =

```

```

{
    D3D_FEATURE_LEVEL_11_1,
    D3D_FEATURE_LEVEL_11_0,
    D3D_FEATURE_LEVEL_10_1,
    D3D_FEATURE_LEVEL_10_0,
    D3D_FEATURE_LEVEL_9_3,
    D3D_FEATURE_LEVEL_9_2,
    D3D_FEATURE_LEVEL_9_1
};

// Create the Direct3D 11 API device object and a corresponding context.
ComPtr<ID3D11Device> device;
ComPtr<ID3D11DeviceContext> context;

HRESULT hr = D3D11CreateDevice(
    nullptr,                // Specify nullptr to use the default
    D3D_DRIVER_TYPE_HARDWARE, // Create a device using the hardware
    0,                      // Should be 0 unless the driver is
    D3D_DRIVER_TYPE_SOFTWARE, // graphics driver.
    creationFlags,          // Set debug and Direct2D compatibility
    featureLevels,          // List of feature levels this app can
    ARRAYSIZE(featureLevels), // support.
    D3D11_SDK_VERSION,     // Size of the list above.
    &device,               // Always set this to D3D11_SDK_VERSION for
    &m_d3dFeatureLevel,    // UWP apps.
    &context,             // Returns the Direct3D device created.
    &context              // Returns feature level of device created.
    );                  // Returns the device immediate context.

if (FAILED(hr))
{
    // If the initialization fails, fall back to the WARP device.
    // For more information on WARP, see:
    // https://go.microsoft.com/fwlink/?LinkId=286690
    DX::ThrowIfFailed(
        D3D11CreateDevice(
            nullptr,
            D3D_DRIVER_TYPE_WARP, // Create a WARP device instead of a
            0,
            creationFlags,
            featureLevels,
            ARRAYSIZE(featureLevels),
            D3D11_SDK_VERSION,
            &device,
            &m_d3dFeatureLevel,
            &context
        )
    );
}

```

```
// Store pointers to the Direct3D 11.1 API device and immediate context.
DX::ThrowIfFailed(
    device.As(&m_d3dDevice)
);

DX::ThrowIfFailed(
    context.As(&m_d3dContext)
);
```

The **DeviceResources::CreateDeviceResources** method then creates the Direct2D device. Direct2D uses Microsoft DirectX Graphics Infrastructure (DXGI) to interoperate with Direct3D. DXGI enables video memory surfaces to be shared between graphics runtimes. Marble Maze uses the underlying DXGI device from the Direct3D device to create the Direct2D device from the Direct2D factory.

C++

```
// Create the Direct2D device object and a corresponding context.
ComPtr<IDXGIDevice3> dxgiDevice;
DX::ThrowIfFailed(
    m_d3dDevice.As(&dxgiDevice)
);

DX::ThrowIfFailed(
    m_d2dFactory->CreateDevice(dxgiDevice.Get(), &m_d2dDevice)
);

DX::ThrowIfFailed(
    m_d2dDevice->CreateDeviceContext(
        D2D1_DEVICE_CONTEXT_OPTIONS_NONE,
        &m_d2dContext
    )
);
```

For more information about DXGI and interoperability between Direct2D and Direct3D, see [DXGI Overview](#) and [Direct2D and Direct3D Interoperability Overview](#).

Associating Direct3D with the view

The **DeviceResources::CreateWindowSizeDependentResources** method creates the graphics resources that depend on a given window size such as the swap chain and Direct3D and Direct2D render targets. One important way that a DirectX UWP app differs from a desktop app is how the swap chain is associated with the output window. A swap chain is responsible for displaying the buffer to which the device renders on the monitor. [Marble Maze application structure](#) describes how the windowing system for a UWP app differs from a desktop app. Because a UWP app does not work with **HWND**

objects, Marble Maze must use the [IDXGIFactory2::CreateSwapChainForCoreWindow](#) method to associate the device output to the view. The following example shows the part of the **DeviceResources::CreateWindowSizeDependentResources** method that creates the swap chain.

C++

```
// Obtain the final swap chain for this window from the DXGI factory.
DX::ThrowIfFailed(
    dxgiFactory->CreateSwapChainForCoreWindow(
        m_d3DDevice.Get(),
        reinterpret_cast<IUnknown*>(m_window.Get()),
        &swapChainDesc,
        nullptr,
        &m_swapChain
    )
);
```

To minimize power consumption, which is important to do on battery-powered devices such as laptops and tablets, the **DeviceResources::CreateWindowSizeDependentResources** method calls the [IDXGIDevice1::SetMaximumFrameLatency](#) method to ensure that the game is rendered only after the vertical blank. Synchronizing with the vertical blank is described in greater detail in the section [Presenting the scene](#) in this document.

C++

```
// Ensure that DXGI does not queue more than one frame at a time. This both
// reduces latency and ensures that the application will only render after
// each
// VSync, minimizing power consumption.
DX::ThrowIfFailed(
    dxgiDevice->SetMaximumFrameLatency(1)
);
```

The **DeviceResources::CreateWindowSizeDependentResources** method initializes graphics resources in a way that works for most games.

ⓘ Note

The term *view* has a different meaning in the Windows Runtime than it has in Direct3D. In the Windows Runtime, a view refers to the collection of user interface settings for an app, including the display area and the input behaviors, plus the thread it uses for processing. You specify the configuration and settings you need when you create a view. The process of setting up the app view is described in

Marble Maze application structure. In Direct3D, the term view has multiple meanings. A resource view defines the subresources that a resource can access. For example, when a texture object is associated with a shader resource view, that shader can later access the texture. One advantage of a resource view is that you can interpret data in different ways at different stages in the rendering pipeline. For more information about resource views, see **Resource Views**. When used in the context of a view transform or view transform matrix, view refers to the location and orientation of the camera. A view transform relocates objects in the world around the camera's position and orientation. For more information about view transforms, see **View Transform (Direct3D 9)**. How Marble Maze uses resource and matrix views is described in greater detail in this topic.

Loading scene resources

Marble Maze uses the **BasicLoader** class, which is declared in **BasicLoader.h**, to load textures and shaders. Marble Maze uses the **SDKMesh** class to load the 3D meshes for the maze and the marble.

To ensure a responsive app, Marble Maze loads scene resources asynchronously, or in the background. As assets load in the background, your game can respond to window events. This process is explained in greater detail in [Loading game assets in the background](#) in this guide.

Loading the 2D overlay and user interface

In Marble Maze, the overlay is the image that appears at the top of the screen. The overlay always appears in front of the scene. In Marble Maze, the overlay contains the Windows logo and the text string **DirectX Marble Maze game sample**. The management of the overlay is performed by the **SampleOverlay** class, which is defined in **SampleOverlay.h**. Although we use the overlay as part of the Direct3D samples, you can adapt this code to display any image that appears in front of your scene.

One important aspect of the overlay is that, because its contents do not change, the **SampleOverlay** class draws, or caches, its contents to an **ID2D1Bitmap1** object during initialization. At draw time, the **SampleOverlay** class only has to draw the bitmap to the screen. In this way, expensive routines such as text drawing do not have to be performed for every frame.

The user interface (UI) consists of 2D components, such as menus and heads-up displays (HUDs), which appear in front of your scene. Marble Maze defines the following UI elements:

- Menu items that enable the user to start the game or view high scores.
- A timer that counts down for three seconds before play begins.
- A timer that tracks the elapsed play time.
- A table that lists the fastest finish times.
- Text that reads **Paused** when the game is paused.

Marble Maze defines game-specific UI elements in **UserInterface.h**. Marble Maze defines the **ElementBase** class as a base type for all UI elements. The **ElementBase** class defines attributes such as the size, position, alignment, and visibility of a UI element. It also controls how elements are updated and rendered.

C++

```
class ElementBase
{
public:
    virtual void Initialize() { }
    virtual void Update(float timeTotal, float timeDelta) { }
    virtual void Render() { }

    void SetAlignment(AlignType horizontal, AlignType vertical);
    virtual void SetContainer(const D2D1_RECT_F& container);
    void SetVisible(bool visible);

    D2D1_RECT_F GetBounds();

    bool IsVisible() const { return m_visible; }

protected:
    ElementBase();

    virtual void CalculateSize() { }

    Alignment        m_alignment;
    D2D1_RECT_F       m_container;
    D2D1_SIZE_F       m_size;
    bool              m_visible;
};
```

By providing a common base class for UI elements, the **UserInterface** class, which manages the user interface, need only hold a collection of **ElementBase** objects, which simplifies UI management and provides a user interface manager that is reusable. Marble Maze defines types that derive from **ElementBase** that implement game-specific

behaviors. For example, **HighScoreTable** defines the behavior for the high score table. For more info about these types, refer to the source code.

ⓘ Note

Because XAML enables you to more easily create complex user interfaces, like those found in simulation and strategy games, consider whether to use XAML to define your UI. For info about how to develop a user interface in XAML in a DirectX UWP game, see **Extend the game sample**, which refers to the DirectX 3D shooting game sample.

Loading shaders

Marble Maze uses the **BasicLoader::LoadShader** method to load a shader from a file.

Shaders are the fundamental unit of GPU programming in games today. Nearly all 3D graphics processing is driven through shaders, whether it is model transformation and scene lighting, or more complex geometry processing, from character skinning to tessellation. For more information about the shader programming model, see [HLSL](#).

Marble Maze uses vertex and pixel shaders. A vertex shader always operates on one input vertex and produces one vertex as output. A pixel shader takes numeric values, texture data, interpolated per-vertex values, and other data to produce a pixel color as output. Because a shader transforms one element at a time, graphics hardware that provides multiple shader pipelines can process sets of elements in parallel. The number of parallel pipelines that are available to the GPU can be vastly greater than the number that is available to the CPU. Therefore, even basic shaders can greatly improve throughput.

The **MarbleMazeMain::LoadDeferredResources** method loads one vertex shader and one pixel shader after it loads the overlay. The design-time versions of these shaders are defined in **BasicVertexShader.hlsl** and **BasicPixelShader.hlsl**, respectively. Marble Maze applies these shaders to both the ball and the maze during the rendering phase.

The Marble Maze project includes both .hlsl (the design-time format) and .cso (the run-time format) versions of the shader files. At build time, Visual Studio uses the fxc.exe effect-compiler to compile your .hlsl source file into a .cso binary shader. For more information about the effect-compiler tool, see [Effect-Compiler Tool](#).

The vertex shader uses the supplied model, view and projection matrices to transform the input geometry. Position data from the input geometry is transformed and output twice: once in screen space, which is necessary for rendering, and again in world space to enable the pixel shader to perform lighting calculations. The surface normal vector is transformed to world space, which is also used by the pixel shader for lighting. The texture coordinates are passed through unchanged to the pixel shader.

hlsl

```
sPSInput main(sVSInput input)
{
    sPSInput output;
    float4 temp = float4(input.pos, 1.0f);
    temp = mul(temp, model);
    output.worldPos = temp.xyz / temp.w;
    temp = mul(temp, view);
    temp = mul(temp, projection);
    output.pos = temp;
    output.tex = input.tex;
    output.norm = mul(float4(input.norm, 0.0f), model).xyz;
    return output;
}
```

The pixel shader receives the output of the vertex shader as input. This shader performs lighting calculations to mimic a soft-edged spotlight that hovers over the maze and is aligned with the position of the marble. Lighting is strongest for surfaces that point directly toward the light. The diffuse component tapers off to zero as the surface normal becomes perpendicular to the light, and the ambient term diminishes as the normal points away from the light. Points closer to the marble (and therefore closer to the center of the spotlight) are lit more strongly. However, lighting is modulated for points underneath the marble to simulate a soft shadow. In a real environment, an object like the white marble would diffusely reflect the spotlight onto other objects in the scene. This is approximated for the surfaces that are in view of the bright half of the marble. The additional illumination factors are in relative angle and distance to the marble. The resulting pixel color is a composition of the sampled texture with the result of the lighting calculations.

hlsl

```
float4 main(sPSInput input) : SV_TARGET
{
    float3 lightDirection = float3(0, 0, -1);
    float3 ambientColor = float3(0.43, 0.31, 0.24);
    float3 lightColor = 1 - ambientColor;
    float spotRadius = 50;

    // Basic ambient (Ka) and diffuse (Kd) lighting from above.
```

```

float3 N = normalize(input.norm);
float NdotL = dot(N, lightDirection);
float Ka = saturate(NdotL + 1);
float Kd = saturate(NdotL);

// Spotlight.
float3 vec = input.worldPos - marblePosition;
float dist2D = sqrt(dot(vec.xy, vec.xy));
Kd = Kd * saturate(spotRadius / dist2D);

// Shadowing from ball.
if (input.worldPos.z > marblePosition.z)
    Kd = Kd * saturate(dist2D / (marbleRadius * 1.5));

// Diffuse reflection of light off ball.
float dist3D = sqrt(dot(vec, vec));
float3 V = normalize(vec);
Kd += saturate(dot(-V, N)) * saturate(dot(V, lightDirection))
    * saturate(marbleRadius / dist3D);

// Final composite.
float4 diffuseTexture = Texture.Sample(Sampler, input.tex);
float3 color = diffuseTexture.rgb * ((ambientColor * Ka) + (lightColor *
Kd));
return float4(color * lightStrength, diffuseTexture.a);
}

```

Warning

The compiled pixel shader contains 32 arithmetic instructions and 1 texture instruction. This shader should perform well on desktop computers or higher-performing tablets. However, some computers might not be able to process this shader and still provide an interactive frame rate. Consider the typical hardware of your target audience and design your shaders to meet the capabilities of that hardware.

The **MarbleMazeMain::LoadDeferredResources** method uses the **BasicLoader::LoadShader** method to load the shaders. The following example loads the vertex shader. The run-time format for this shader is **BasicVertexShader.cso**. The **m_vertexShader** member variable is an [ID3D11VertexShader](#) object.

C++

```

BasicLoader^ loader = ref new BasicLoader(m_deviceResources-
>GetD3DDevice());

```

```

D3D11_INPUT_ELEMENT_DESC layoutDesc [] =
{
    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "TANGENT", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 32,
D3D11_INPUT_PER_VERTEX_DATA, 0 },
};
m_vertexStride = 44; // must set this to match the size of layoutDesc above

Platform::String^ vertexShaderName = L"BasicVertexShader.cso";
loader->LoadShader(
    vertexShaderName,
    layoutDesc,
    ARRAYSIZE(layoutDesc),
    &m_vertexShader,
    &m_inputLayout
);

```

The `m_inputLayout` member variable is an [ID3D11InputLayout](#) object. The input-layout object encapsulates the input state of the input assembler (IA) stage. One job of the IA stage is to make shaders more efficient by using system-generated values, also known as *semantics*, to process only those primitives or vertices that have not already been processed.

Use the [ID3D11Device::CreateInputLayout](#) method to create an input-layout from an array of input-element descriptions. The array contains one or more input elements; each input element describes one vertex-data element from one vertex buffer. The entire set of input-element descriptions describes all of the vertex-data elements from all of the vertex buffers that will be bound to the IA stage.

`layoutDesc` in the above code snippet shows the layout description that Marble Maze uses. The layout description describes a vertex buffer that contains four vertex-data elements. The important parts of each entry in the array are the semantic name, data format, and byte offset. For example, the **POSITION** element specifies the vertex position in object space. It starts at byte offset 0 and contains three floating-point components (for a total of 12 bytes). The **NORMAL** element specifies the normal vector. It starts at byte offset 12 because it appears directly after **POSITION** in the layout, which requires 12 bytes. The **NORMAL** element contains a four-component, 32-bit unsigned-integer.

Compare the input layout with the **sVSInput** structure that is defined by the vertex shader, as shown in the following example. The **sVSInput** structure defines the

POSITION, **NORMAL**, and **TEXCOORD0** elements. The DirectX runtime maps each element in the layout to the input structure that is defined by the shader.

hlsl

```
struct sVSInput
{
    float3 pos : POSITION;
    float3 norm : NORMAL;
    float2 tex : TEXCOORD0;
};

struct sPSInput
{
    float4 pos : SV_POSITION;
    float3 norm : NORMAL;
    float2 tex : TEXCOORD0;
    float3 worldPos : TEXCOORD1;
};

sPSInput main(sVSInput input)
{
    sPSInput output;
    float4 temp = float4(input.pos, 1.0f);
    temp = mul(temp, model);
    output.worldPos = temp.xyz / temp.w;
    temp = mul(temp, view);
    temp = mul(temp, projection);
    output.pos = temp;
    output.tex = input.tex;
    output.norm = mul(float4(input.norm, 0.0f), model).xyz;
    return output;
}
```

The document [Semantics](#) describes each of the available semantics in greater detail.

ⓘ Note

In a layout, you can specify additional components that are not used to enable multiple shaders to share the same layout. For example, the **TANGENT** element is not used by the shader. You can use the **TANGENT** element if you want to experiment with techniques such as normal mapping. By using normal mapping, also known as bump mapping, you can create the effect of bumps on the surfaces of objects. For more information about bump mapping, see [Bump Mapping \(Direct3D 9\)](#).

For more information about the input assembly stage, see [Input-Assembler Stage](#) and [Getting Started with the Input-Assembler Stage](#).

The process of using the vertex and pixel shaders to render the scene are described in the section [Rendering the scene](#) later in this document.

Creating the constant buffer

Direct3D buffer groups a collection of data. A constant buffer is a kind of buffer that you can use to pass data to shaders. Marble Maze uses a constant buffer to hold the model (or world) view, and the projection matrices for the active scene object.

The following example shows how the **MarbleMazeMain::LoadDeferredResources** method creates a constant buffer that will later hold matrix data. The example creates a **D3D11_BUFFER_DESC** structure that uses the **D3D11_BIND_CONSTANT_BUFFER** flag to specify usage as a constant buffer. This example then passes that structure to the [ID3D11Device::CreateBuffer](#) method. The **m_constantBuffer** variable is an [ID3D11Buffer](#) object.

C++

```
// Create the constant buffer for updating model and camera data.
D3D11_BUFFER_DESC constantBufferDesc = {0};

// Multiple of 16 bytes
constantBufferDesc.ByteWidth = ((sizeof(ConstantBuffer) + 15) / 16) * 16;

constantBufferDesc.Usage                = D3D11_USAGE_DEFAULT;
constantBufferDesc.BindFlags            = D3D11_BIND_CONSTANT_BUFFER;
constantBufferDesc.CPUAccessFlags      = 0;
constantBufferDesc.MiscFlags           = 0;

// This will not be used as a structured buffer, so this parameter is
// ignored.
constantBufferDesc.StructureByteStride = 0;

DX::ThrowIfFailed(
    m_deviceResources->GetD3DDevice()->CreateBuffer(
        &constantBufferDesc,
        nullptr,    // leave the buffer uninitialized
        &m_constantBuffer
    )
);
```

The **MarbleMazeMain::Update** method later updates **ConstantBuffer** objects, one for the maze and one for the marble. The **MarbleMazeMain::Render** method then binds

each **ConstantBuffer** object to the constant buffer before each object is rendered. The following example shows the **ConstantBuffer** structure, which is in **MarbleMazeMain.h**.

C++

```
// Describes the constant buffer that draws the meshes.
struct ConstantBuffer
{
    XMFLOAT4X4 model;
    XMFLOAT4X4 view;
    XMFLOAT4X4 projection;

    XMFLOAT3 marblePosition;
    float marbleRadius;
    float lightStrength;
};
```

To better understand how constant buffers map to shader code, compare the **ConstantBuffer** structure in **MarbleMazeMain.h** to the **ConstantBuffer** constant buffer that is defined by the vertex shader in **BasicVertexShader.hls**:

hls

```
cbuffer ConstantBuffer : register(b0)
{
    matrix model;
    matrix view;
    matrix projection;
    float3 marblePosition;
    float marbleRadius;
    float lightStrength;
};
```

The layout of the **ConstantBuffer** structure matches the **cbuffer** object. The **cbuffer** variable specifies register b0, which means that the constant buffer data is stored in register 0. The **MarbleMazeMain::Render** method specifies register 0 when it activates the constant buffer. This process is described in greater detail later in this document.

For more information about constant buffers, see [Introduction to Buffers in Direct3D 11](#). For more information about the register keyword, see [register](#).

Loading meshes

Marble Maze uses SDK-Mesh as the run-time format because this format provides a basic way to load mesh data for sample applications. For production use, you should use a mesh format that meets the specific requirements of your game.

The **MarbleMazeMain::LoadDeferredResources** method loads mesh data after it loads the vertex and pixel shaders. A mesh is a collection of vertex data that often includes information such as positions, normal data, colors, materials, and texture coordinates. Meshes are typically created in 3D authoring software and maintained in files that are separate from application code. The marble and the maze are two examples of meshes that the game uses.

Marble Maze uses the **SDKMesh** class to manage meshes. This class is declared in **SDKMesh.h**. **SDKMesh** provides methods to load, render, and destroy mesh data.

Important

Marble Maze uses the SDK-Mesh format and provides the **SDKMesh** class for illustration only. Although the SDK-Mesh format is useful for learning, and for creating prototypes, it is a very basic format that might not meet the requirements of most game development. We recommend that you use a mesh format that meets the specific requirements of your game.

The following example shows how the **MarbleMazeMain::LoadDeferredResources** method uses the **SDKMesh::Create** method to load mesh data for the maze and for the ball.

C++

```
// Load the meshes.
DX::ThrowIfFailed(
    m_mazeMesh.Create(
        m_deviceResources->GetD3DDevice(),
        L"Media\\Models\\maze1.sdkmesh",
        false
    )
);

DX::ThrowIfFailed(
    m_marbleMesh.Create(
        m_deviceResources->GetD3DDevice(),
        L"Media\\Models\\marble2.sdkmesh",
        false
    )
);
```

Loading collision data

Although this section does not focus on how Marble Maze implements the physics simulation between the marble and the maze, note that mesh geometry for the physics system is read when the meshes are loaded.

C++

```
// Extract mesh geometry for the physics system.
DX::ThrowIfFailed(
    ExtractTrianglesFromMesh(
        m_mazeMesh,
        "Mesh_walls",
        m_collision.m_wallTriList
    )
);

DX::ThrowIfFailed(
    ExtractTrianglesFromMesh(
        m_mazeMesh,
        "Mesh_Floor",
        m_collision.m_groundTriList
    )
);

DX::ThrowIfFailed(
    ExtractTrianglesFromMesh(
        m_mazeMesh,
        "Mesh_floorSides",
        m_collision.m_floorTriList
    )
);

m_physics.SetCollision(&m_collision);
float radius = m_marbleMesh.GetMeshBoundingBoxExtents(0).x / 2;
m_physics.SetRadius(radius);
```

The way that you load collision data largely depends on the run-time format that you use. For more information about how Marble Maze loads the collision geometry from an SDK-Mesh file, see the **MarbleMazeMain::ExtractTrianglesFromMesh** method in the source code.

Updating game state

Marble Maze separates game logic from rendering logic by first updating all scene objects before rendering them.

[Marble Maze application structure](#) describes the main game loop. Updating the scene, which is part of the game loop, happens after Windows events and input are processed

and before the scene is rendered. The **MarbleMazeMain::Update** method handles the update of the UI and the game.

Updating the user interface

The **MarbleMazeMain::Update** method calls the **UserInterface::Update** method to update the state of the UI.

C++

```
UserInterface::GetInstance().Update(  
    static_cast<float>(m_timer.GetTotalSeconds()),  
    static_cast<float>(m_timer.GetElapsedSeconds()));
```

The **UserInterface::Update** method updates each element in the UI collection.

C++

```
void UserInterface::Update(float timeTotal, float timeDelta)  
{  
    for (auto iter = m_elements.begin(); iter != m_elements.end(); ++iter)  
    {  
        (*iter)->Update(timeTotal, timeDelta);  
    }  
}
```

Classes that derive from **ElementBase** (defined in **UserInterface.h**) implement the **Update** method to perform specific behaviors. For example, the **StopwatchTimer::Update** method updates the elapsed time by the provided amount and updates the text that it later displays.

C++

```
void StopwatchTimer::Update(float timeTotal, float timeDelta)  
{  
    if (m_active)  
    {  
        m_elapsedTime += timeDelta;  
  
        WCHAR buffer[16];  
        GetFormattedTime(buffer);  
        SetText(buffer);  
    }  
  
    TextElement::Update(timeTotal, timeDelta);  
}
```

Updating the scene

The **MarbleMazeMain::Update** method updates the game based on the current state of the state machine (the **GameState**, stored in **m_gameState**). When the game is in the active state (**GameState::InGameActive**), Marble Maze updates the camera to follow the marble, updates the view matrix part of the constant buffers, and updates the physics simulation.

The following example shows how the **MarbleMazeMain::Update** method updates the position of the camera. Marble Maze uses the **m_resetCamera** variable to flag that the camera must be reset to be located directly above the marble. The camera is reset when the game starts or the marble falls through the maze. When the main menu or high score display screen is active, the camera is set at a constant location. Otherwise, Marble Maze uses the *timeDelta* parameter to interpolate the position of the camera between its current and target positions. The target position is slightly above and in front of the marble. Using the elapsed frame time enables the camera to gradually follow, or chase, the marble.

C++

```
static float eyeDistance = 200.0f;
static XMFLOAT3A eyePosition = XMFLOAT3A(0, 0, 0);

// Gradually move the camera above the marble.
XMFLOAT3A targetEyePosition;
XMStoreFloat3A(
    &targetEyePosition,
    XMLoadFloat3A(&marblePosition) - (XMLoadFloat3A(&g) * eyeDistance));

if (m_resetCamera)
{
    eyePosition = targetEyePosition;
    m_resetCamera = false;
}
else
{
    XMStoreFloat3A(
        &eyePosition,
        XMLoadFloat3A(&eyePosition)
            + ((XMLoadFloat3A(&targetEyePosition) -
XMLoadFloat3A(&eyePosition))
                * min(1, static_cast<float>(m_timer.GetElapsedSeconds()) *
8)
            )
    );
}

// Look at the marble.
if ((m_gameState == GameState::MainMenu) || (m_gameState ==
```

```

GameState::HighScoreDisplay))
{
    // Override camera position for menus.
    XMStoreFloat3A(
        &eyePosition,
        XMLoadFloat3A(&marblePosition) + XMVectorSet(75.0f, -150.0f, -75.0f,
0.0f));

    m_camera->SetViewParameters(
        eyePosition,
        marblePosition,
        XMFLOAT3(0.0f, 0.0f, -1.0f));
}
else
{
    m_camera->SetViewParameters(eyePosition, marblePosition, XMFLOAT3(0.0f,
1.0f, 0.0f));
}

```

The following example shows how the **MarbleMazeMain::Update** method updates the constant buffers for the marble and the maze. The maze's model, or world, matrix always remains the identity matrix. Except for the main diagonal, whose elements are all ones, the identity matrix is a square matrix composed of zeros. The marble's model matrix is based on its position matrix times its rotation matrix.

C++

```

// Update the model matrices based on the simulation.
XMStoreFloat4x4(&m_mazeConstantBufferData.model, XMMatrixIdentity());

XMStoreFloat4x4(
    &m_marbleConstantBufferData.model,
    XMMatrixTranspose(
        XMMatrixMultiply(
            marbleRotationMatrix,
            XMMatrixTranslationFromVector(XMLoadFloat3A(&marblePosition))
        )
    )
);

// Update the view matrix based on the camera.
XMFLOAT4X4 view;
m_camera->GetViewMatrix(&view);
m_mazeConstantBufferData.view = view;
m_marbleConstantBufferData.view = view;

```

For information about how the **MarbleMazeMain::Update** method reads user input and simulates the motion of the marble, see [Adding input and interactivity to the Marble Maze sample](#).

Rendering the scene

When a scene is rendered, these steps are typically included.

1. Set the current render target depth-stencil buffer.
2. Clear the render and stencil views.
3. Prepare the vertex and pixel shaders for drawing.
4. Render the 3D objects in the scene.
5. Render any 2D object that you want to appear in front of the scene.
6. Present the rendered image to the monitor.

The **MarbleMazeMain::Render** method binds the render target and depth stencil views, clears those views, draws the scene, and then draws the overlay.

Preparing the render targets

Before you render your scene, you must set the current render target depth-stencil buffer. If your scene is not guaranteed to draw over every pixel on the screen, also clear the render and stencil views. Marble Maze clears the render and stencil views on every frame to ensure that there are no visible artifacts from the previous frame.

The following example shows how the **MarbleMazeMain::Render** method calls the [ID3D11DeviceContext::OMSetRenderTargets](#) method to set the render target and the depth-stencil buffer as the current ones.

C++

```
auto context = m_deviceResources->GetD3DDeviceContext();

// Reset the viewport to target the whole screen.
auto viewport = m_deviceResources->GetScreenViewport();
context->RSSetViewports(1, &viewport);

// Reset render targets to the screen.
ID3D11RenderTargetView *const targets[1] =
    { m_deviceResources->GetBackBufferRenderTargetView() };

context->OMSetRenderTargets(1, targets, m_deviceResources->
    GetDepthStencilView());

// Clear the back buffer and depth stencil view.
context->ClearRenderTargetView(
    m_deviceResources->GetBackBufferRenderTargetView(),
    DirectX::Colors::Black);

context->ClearDepthStencilView(
    m_deviceResources->GetDepthStencilView(),
```

```
D3D11_CLEAR_DEPTH | D3D11_CLEAR_STENCIL,  
1.0f,  
0);
```

The [ID3D11RenderTargetView](#) and [ID3D11DepthStencilView](#) interfaces support the texture view mechanism that is provided by Direct3D 10 and later. For more information about texture views, see [Texture Views \(Direct3D 10\)](#). The [OMSetRenderTargets](#) method prepares the output-merger stage of the Direct3D pipeline. For more information about the output-merger stage, see [Output-Merger Stage](#).

Preparing the vertex and pixel shaders

Before you render the scene objects, perform the following steps to prepare the vertex and pixel shaders for drawing:

1. Set the shader input layout as the current layout.
2. Set the vertex and pixel shaders as the current shaders.
3. Update any constant buffers with data that you have to pass to the shaders.

Important

Marble Maze uses one pair of vertex and pixel shaders for all 3D objects. If your game uses more than one pair of shaders, you must perform these steps each time you draw objects that use different shaders. To reduce the overhead that is associated with changing the shader state, we recommend that you group render calls for all objects that use the same shaders.

The section [Loading shaders](#) in this document describes how the input layout is created when the vertex shader is created. The following example shows how the **MarbleMazeMain::Render** method uses the [ID3D11DeviceContext::IASetInputLayout](#) method to set this layout as the current layout.

C++

```
m_deviceResources->GetD3DDeviceContext()-  
>IASetInputLayout(m_inputLayout.Get());
```

The following example shows how the **MarbleMazeMain::Render** method uses the [ID3D11DeviceContext::VSSetShader](#) and [ID3D11DeviceContext::PSSetShader](#) methods to set the vertex and pixel shaders as the current shaders, respectively.

C++

```
// Set the vertex shader stage state.
m_deviceResources->GetD3DDeviceContext()->VSSetShader(
    m_vertexShader.Get(),    // use this vertex shader
    nullptr,                 // don't use shader linkage
    0);                      // don't use shader linkage

m_deviceResources->GetD3DDeviceContext()->PSSetShader(
    m_pixelShader.Get(),     // use this pixel shader
    nullptr,                 // don't use shader linkage
    0);                      // don't use shader linkage

m_deviceResources->GetD3DDeviceContext()->PSSetSamplers(
    0,                       // starting at the first sampler slot
    1,                       // set one sampler binding
    m_sampler.GetAddressOf()); // to use this sampler
```

After **MarbleMazeMain::Render** sets the shaders and their input layout, it uses the [ID3D11DeviceContext::UpdateSubresource](#) method to update the constant buffer with the model, view, and projection matrices for the maze. The **UpdateSubresource** method copies the matrix data from CPU memory to GPU memory. Recall that the model and view components of the **ConstantBuffer** structure are updated in the **MarbleMazeMain::Update** method. The **MarbleMazeMain::Render** method then calls the [ID3D11DeviceContext::VSSetConstantBuffers](#) and [ID3D11DeviceContext::PSSetConstantBuffers](#) methods to set this constant buffer as the current one.

C++

```
// Update the constant buffer with the new data.
m_deviceResources->GetD3DDeviceContext()->UpdateSubresource(
    m_constantBuffer.Get(),
    0,
    nullptr,
    &m_mazeConstantBufferData,
    0,
    0);

m_deviceResources->GetD3DDeviceContext()->VSSetConstantBuffers(
    0,                               // starting at the first constant
buffer slot
    1,                               // set one constant buffer binding
    m_constantBuffer.GetAddressOf()); // to use this buffer

m_deviceResources->GetD3DDeviceContext()->PSSetConstantBuffers(
    0,                               // starting at the first constant
buffer slot
```

```
1, // set one constant buffer binding
m_constantBuffer.GetAddressOf()); // to use this buffer
```

The **MarbleMazeMain::Render** method performs similar steps to prepare the marble to be rendered.

Rendering the maze and the marble

After you activate the current shaders, you can draw your scene objects. The **MarbleMazeMain::Render** method calls the **SDKMesh::Render** method to render the maze mesh.

C++

```
m_mazeMesh.Render(
    m_deviceResources->GetD3DDeviceContext(),
    0,
    INVALID_SAMPLER_SLOT,
    INVALID_SAMPLER_SLOT);
```

The **MarbleMazeMain::Render** method performs similar steps to render the marble.

As mentioned earlier in this document, the **SDKMesh** class is provided for demonstration purposes, but we do not recommend it for use in a production-quality game. However, notice that the **SDKMesh::RenderMesh** method, which is called by **SDKMesh::Render**, uses the [ID3D11DeviceContext::IASetVertexBuffers](#) and [ID3D11DeviceContext::IASetIndexBuffer](#) methods to set the current vertex and index buffers that define the mesh, and the [ID3D11DeviceContext::DrawIndexed](#) method to draw the buffers. For more information about how to work with vertex and index buffers, see [Introduction to Buffers in Direct3D 11](#).

Drawing the user interface and overlay

After drawing 3D scene objects, Marble Maze draws the 2D UI elements that appear in front of the scene.

The **MarbleMazeMain::Render** method ends by drawing the user interface and the overlay.

C++

```
// Draw the user interface and the overlay.
UserInterface::GetInstance().Render(m_deviceResources->
    GetOrientationTransform2D());
```

```
m_deviceResources->GetD3DDeviceContext()->BeginEventInt(L"Render Overlay",
0);
m_sampleOverlay->Render();
m_deviceResources->GetD3DDeviceContext()->EndEvent();
```

The **UserInterface::Render** method uses an **ID2D1DeviceContext** object to draw the UI elements. This method sets the drawing state, draws all active UI elements, and then restores the previous drawing state.

C++

```
void UserInterface::Render(D2D1::Matrix3x2F orientation2D)
{
    m_d2dContext->SaveDrawingState(m_stateBlock.Get());
    m_d2dContext->BeginDraw();
    m_d2dContext->SetTransform(orientation2D);

    m_d2dContext->SetTextAntialiasMode(D2D1_TEXT_ANTIALIAS_MODE_GRAYSCALE);

    for (auto iter = m_elements.begin(); iter != m_elements.end(); ++iter)
    {
        if ((*iter)->IsVisible())
            (*iter)->Render();
    }

    // We ignore D2DERR_RECREATE_TARGET here. This error indicates that the
    device
    // is lost. It will be handled during the next call to Present.
    HRESULT hr = m_d2dContext->EndDraw();
    if (hr != D2DERR_RECREATE_TARGET)
    {
        DX::ThrowIfFailed(hr);
    }

    m_d2dContext->RestoreDrawingState(m_stateBlock.Get());
}
```

The **SampleOverlay::Render** method uses a similar technique to draw the overlay bitmap.

Presenting the scene

After drawing all 2D and 3D scene objects, Marble Maze presents the rendered image to the monitor. It synchronizes drawing to the vertical blank to ensure that time is not spent time drawing frames that will never be actually shown on the display. Marble Maze also handles device changes when it presents the scene.

After the `MarbleMazeMain::Render` method returns, the game loop calls the `DX::DeviceResources::Present` method to send the rendered image to the monitor or display. The `DX::DeviceResources::Present` method calls `IDXGISwapChain::Present` to perform the present operation, as shown in the following example:

C++

```
// The first argument instructs DXGI to block until VSync, putting the
// application
// to sleep until the next VSync. This ensures we don't waste any cycles
// rendering
// frames that will never be displayed to the screen.
HRESULT hr = m_swapChain->Present(1, 0);
```

In this example, `m_swapChain` is an `IDXGISwapChain1` object. The initialization of this object is described in the section [Initializing Direct3D and Direct2D](#) in this document.

The first parameter to `IDXGISwapChain::Present`, *SyncInterval*, specifies the number of vertical blanks to wait before presenting the frame. Marble Maze specifies 1 so that it waits until the next vertical blank.

The `IDXGISwapChain::Present` method returns an error code that indicates that the device was removed or otherwise failed. In this case, Marble Maze reinitializes the device.

C++

```
// If the device was removed either by a disconnection or a driver upgrade,
// we
// must recreate all device resources.
if (hr == DXGI_ERROR_DEVICE_REMOVED)
{
    HandleDeviceLost();
}
else
{
    DX::ThrowIfFailed(hr);
}
```

Next steps

Read [Adding input and interactivity to the Marble Maze sample](#) for information about some of the key practices to keep in mind when you work with input devices. This document discusses how Marble Maze supports touch, accelerometer, game controllers, and mouse input.

Related topics

- [Adding input and interactivity to the Marble Maze sample](#)
- [Marble Maze application structure](#)
- [Developing Marble Maze, a UWP game in C++ and DirectX](#)

Adding input and interactivity to the Marble Maze sample

Article • 10/20/2022

Universal Windows Platform (UWP) games run on a wide variety of devices, such as desktop computers, laptops, and tablets. A device can have a plethora of input and control mechanisms. This document describes the key practices to keep in mind when you work with input devices and shows how Marble Maze applies these practices.

ⓘ Note

The sample code that corresponds to this document is found in the [DirectX Marble Maze game sample](#).

Here are some of the key points that this document discusses for when you work with input in your game:

- When possible, support multiple input devices to enable your game to accommodate a wider range of preferences and abilities among your customers. Although game controller and sensor usage is optional, we strongly recommend it to enhance the player experience. We designed the game controller and sensor APIs to help you more easily integrate these input devices.
- To initialize touch, you must register for window events such as when the pointer is activated, released, and moved. To initialize the accelerometer, create a [Windows::Devices::Sensors::Accelerometer](#) object when you initialize the application. A game controller doesn't require initialization.
- For single-player games, consider whether to combine input from all possible controllers. This way, you don't have to track what input comes from which controller. Or, simply track input only from the most recently added controller, as we do in this sample.
- Process Windows events before you process input devices.
- Game controller and accelerometer support polling. That is, you can poll for data when you need it. For touch, record touch events in data structures that are available to your input processing code.
- Consider whether to normalize input values to a common format. Doing so can simplify how input is interpreted by other components of your game, such as

physics simulation, and can make it easier to write games that work on different screen resolutions.

Input devices supported by Marble Maze

Marble Maze supports the game controller, mouse, and touch to select menu items, and the game controller, mouse, touch, and the accelerometer to control game play. Marble Maze uses the [Windows::Gaming::Input](#) APIs to poll the controller for input. Touch enables applications to track and respond to fingertip input. An accelerometer is a sensor that measures the force that is applied along the X, Y, and Z axes. By using the Windows Runtime, you can poll the current state of the accelerometer device, as well as receive touch events through the Windows Runtime event-handling mechanism.

ⓘ Note

This document uses touch to refer to both touch and mouse input and pointer to refer to any device that uses pointer events. Because touch and the mouse use standard pointer events, you can use either device to select menu items and control game play.

ⓘ Note

The package manifest sets **Landscape** as the only supported rotation for the game to prevent the orientation from changing when you rotate the device to roll the marble. To view the package manifest, open **Package.appxmanifest** in the **Solution Explorer** in Visual Studio.

Initializing input devices

The game controller does not require initialization. To initialize touch, you must register for windowing events such as when the pointer is activated (for example, the player presses the mouse button or touches the screen), released, and moved. To initialize the accelerometer, you have to create a [Windows::Devices::Sensors::Accelerometer](#) object when you initialize the application.

The following example shows how the **App::SetWindow** method registers for the [Windows::UI::Core::CoreWindow::PointerPressed](#), [Windows::UI::Core::CoreWindow::PointerReleased](#), and [Windows::UI::Core::CoreWindow::PointerMoved](#) pointer events. These events are registered during application initialization and before the game loop.

These events are handled in a separate thread that invokes the event handlers.

For more information about how the application is initialized, see [Marble Maze application structure](#).

C++

```
window->PointerPressed += ref new TypedEventHandler<CoreWindow^,
PointerEventArgs^>(
    this,
    &App::OnPointerPressed);

window->PointerReleased += ref new TypedEventHandler<CoreWindow^,
PointerEventArgs^>(
    this,
    &App::OnPointerReleased);

window->PointerMoved += ref new TypedEventHandler<CoreWindow^,
PointerEventArgs^>(
    this,
    &App::OnPointerMoved);
```

The **MarbleMazeMain** class also creates a **std::map** object to hold touch events. The key for this map object is a value that uniquely identifies the input pointer. Each key maps to the distance between every touch point and the center of the screen. Marble Maze later uses these values to calculate the amount by which the maze is tilted.

C++

```
typedef std::map<int, XMFLT2> TouchMap;
TouchMap m_touches;
```

The **MarbleMazeMain** class also holds an [Accelerometer](#) object.

C++

```
Windows::Devices::Sensors::Accelerometer^ m_accelerometer;
```

The **Accelerometer** object is initialized in the **MarbleMazeMain** constructor, as shown in the following example. The [Windows::Devices::Sensors::Accelerometer::GetDefault](#)

method returns an instance of the default accelerometer. If there is no default accelerometer, **Accelerometer::GetDefault** returns **nullptr**.

C++

```
// Returns accelerometer ref if there is one; nullptr otherwise.
m_accelerometer = Windows::Devices::Sensors::Accelerometer::GetDefault();
```

Navigating the menus

You can use the mouse, touch, or a game controller to navigate the menus, as follows:

- Use the directional pad to change the active menu item.
- Use touch, the A button, or the Menu button to pick a menu item or close the current menu, such as the high-score table.
- Use the Menu button to pause or resume the game.
- Click on a menu item with the mouse to choose that action.

Tracking game controller input

To keep track of the gamepads currently connected to the device, **MarbleMazeMain** defines a member variable, **m_myGamepads**, which is a collection of [Windows::Gaming::Input::Gamepad](#) objects. This is initialized in the constructor like so:

C++

```
m_myGamepads = ref new Vector<Gamepad^>();

for (auto gamepad : Gamepad::Gamepads)
{
    m_myGamepads->Append(gamepad);
}
```

Additionally, the **MarbleMazeMain** constructor registers events for when gamepads are added or removed:

C++

```
Gamepad::GamepadAdded +=
    ref new EventHandler<Gamepad^>([=](Platform::Object^, Gamepad^ args)
    {
        m_myGamepads->Append(args);
        m_currentGamepadNeedsRefresh = true;
    }));
```

```
Gamepad::GamepadRemoved +=
    ref new EventHandler<Gamepad ^>([=](Platform::Object^, Gamepad^ args)
    {
        unsigned int indexRemoved;

        if (m_myGamepads->IndexOf(args, &indexRemoved))
        {
            m_myGamepads->RemoveAt(indexRemoved);
            m_currentGamepadNeedsRefresh = true;
        }
    });
```

When a gamepad is added, it is added to **m_myGamepads**; when a gamepad is removed, we check if the gamepad is in **m_myGamepads**, and if it is, we remove it. In both cases, we set **m_currentGamepadNeedsRefresh** to **true**, indicating that we need to reassign **m_gamepad**.

Finally, we assign a gamepad to **m_gamepad** and set **m_currentGamepadNeedsRefresh** to **false**:

C++

```
m_gamepad = GetLastGamepad();
m_currentGamepadNeedsRefresh = false;
```

In the **Update** method, we check if **m_gamepad** needs to be reassigned:

C++

```
if (m_currentGamepadNeedsRefresh)
{
    auto mostRecentGamepad = GetLastGamepad();

    if (m_gamepad != mostRecentGamepad)
    {
        m_gamepad = mostRecentGamepad;
    }

    m_currentGamepadNeedsRefresh = false;
}
```

If **m_gamepad** does need to be reassigned, we assign to it the most recently added gamepad, using **GetLastGamepad**, which is defined like so:

C++

```
Gamepad^ MarbleMaze::MarbleMazeMain::GetLastGamepad()
{
```

```

Gamepad^ gamepad = nullptr;

if (m_myGamepads->Size > 0)
{
    gamepad = m_myGamepads->GetAt(m_myGamepads->Size - 1);
}

return gamepad;
}

```

This method simply returns the last gamepad in **m_myGamepads**.

You can connect up to four game controllers to a Windows 10 device. To avoid having to figure out which controller is the active one, we simply only track the most recently added gamepad. If your game supports more than one player, you have to track input for each player separately.

The **MarbleMazeMain::Update** method polls the gamepad for input:

C++

```

if (m_gamepad != nullptr)
{
    m_oldReading = m_newReading;
    m_newReading = m_gamepad->GetCurrentReading();
}

```

We keep track of the input reading we got in the last frame with **m_oldReading**, and the latest input reading with **m_newReading**, which we get by calling [Gamepad::GetCurrentReading](#). This returns a [GamepadReading](#) object, which contains information about the current state of the gamepad.

To check if a button was just pressed or released, we define

MarbleMazeMain::ButtonJustPressed and **MarbleMazeMain::ButtonJustReleased**, which compare button readings from this frame and the last frame. This way, we can perform an action only at the time when a button is initially pressed or released, and not when it's held:

C++

```

bool MarbleMaze::MarbleMazeMain::ButtonJustPressed(GamepadButtons selection)
{
    bool newSelectionPressed = (selection == (m_newReading.Buttons &
selection));
    bool oldSelectionPressed = (selection == (m_oldReading.Buttons &
selection));
    return newSelectionPressed && !oldSelectionPressed;
}

```

```

bool MarbleMaze::MarbleMazeMain::ButtonJustReleased(GamepadButtons
selection)
{
    bool newSelectionReleased =
        (GamepadButtons::None == (m_newReading.Buttons & selection));

    bool oldSelectionReleased =
        (GamepadButtons::None == (m_oldReading.Buttons & selection));

    return newSelectionReleased && !oldSelectionReleased;
}

```

`GamepadButtons` readings are compared using bitwise operations—we check if a button is pressed using *bitwise and* (&). We determine whether a button was just pressed or released by comparing the old reading and the new reading.

Using the above methods, we check if certain buttons have been pressed and perform any corresponding actions that must happen. For example, when the Menu button (`GamepadButtons::Menu`) is pressed, the game state changes from active to paused or paused to active.

C++

```

if (ButtonJustPressed(GamepadButtons::Menu) || m_pauseKeyPressed)
{
    m_pauseKeyPressed = false;

    if (m_gameState == GameState::InGameActive)
    {
        SetGameState(GameState::InGamePaused);
    }
    else if (m_gameState == GameState::InGamePaused)
    {
        SetGameState(GameState::InGameActive);
    }
}

```

We also check if the player presses the View button, in which case we restart the game or clear the high score table:

C++

```

if (ButtonJustPressed(GamepadButtons::View) || m_homeKeyPressed)
{
    m_homeKeyPressed = false;

    if (m_gameState == GameState::InGameActive ||
        m_gameState == GameState::InGamePaused ||

```

```

        m_gameState == GameState::PreGameCountdown)
    {
        SetGameState(GameState::MainMenu);
        m_inGameStopwatchTimer.SetVisible(false);
        m_preGameCountdownTimer.SetVisible(false);
    }
    else if (m_gameState == GameState::HighScoreDisplay)
    {
        m_highScoreTable.Reset();
    }
}

```

If the main menu is active, the active menu item changes when the directional pad is pressed up or down. If the user chooses the current selection, the appropriate UI element is marked as being chosen.

C++

```

// Handle menu navigation.
bool chooseSelection =
    (ButtonJustPressed(GamepadButtons::A)
    || ButtonJustPressed(GamepadButtons::Menu));

bool moveUp = ButtonJustPressed(GamepadButtons::DPadUp);
bool moveDown = ButtonJustPressed(GamepadButtons::DPadDown);

switch (m_gameState)
{
case GameState::MainMenu:
    if (chooseSelection)
    {
        m_audio.PlaySoundEffect(MenuSelectedEvent);
        if (m_startGameButton.GetSelected())
        {
            m_startGameButton.SetPressed(true);
        }
        if (m_highScoreButton.GetSelected())
        {
            m_highScoreButton.SetPressed(true);
        }
    }
    if (moveUp || moveDown)
    {
        m_startGameButton.SetSelected(!m_startGameButton.GetSelected());
        m_highScoreButton.SetSelected(!m_startGameButton.GetSelected());
        m_audio.PlaySoundEffect(MenuChangeEvent);
    }
    break;

case GameState::HighScoreDisplay:
    if (chooseSelection || anyPoints)
    {

```

```

        SetGameState(GameState::MainMenu);
    }
    break;

case GameState::PostGameResults:
    if (chooseSelection || anyPoints)
    {
        SetGameState(GameState::HighScoreDisplay);
    }
    break;

case GameState::InGamePaused:
    if (m_pausedText.IsPressed())
    {
        m_pausedText.SetPressed(false);
        SetGameState(GameState::InGameActive);
    }
    break;
}

```

Tracking touch and mouse input

For touch and mouse input, a menu item is chosen when the user touches or clicks it. The following example shows how the **MarbleMazeMain::Update** method processes pointer input to select menu items. The **m_pointQueue** member variable tracks the locations where the user touched or clicked on the screen. The way in which Marble Maze collects pointer input is described in greater detail later in this document in the section [Processing pointer input](#).

C++

```

// Check whether the user chose a button from the UI.
bool anyPoints = !m_pointQueue.empty();
while (!m_pointQueue.empty())
{
    UserInterface::GetInstance().HitTest(m_pointQueue.front());
    m_pointQueue.pop();
}

```

The **UserInterface::HitTest** method determines whether the provided point is located in the bounds of any UI element. Any UI elements that pass this test are marked as being touched. This method uses the **PointInRect** helper function to determine whether the provided point is located in the bounds of each UI element.

C++

```

void UserInterface::HitTest(D2D1_POINT_2F point)
{

```

```

for (auto iter = m_elements.begin(); iter != m_elements.end(); ++iter)
{
    if (!(*iter)->IsVisible())
        continue;

    TextButton* textButton = dynamic_cast<TextButton*>(*iter);
    if (textButton != nullptr)
    {
        D2D1_RECT_F bounds = (*iter)->GetBounds();
        textButton->SetPressed(PointInRect(point, bounds));
    }
}
}

```

Updating the game state

After the **MarbleMazeMain::Update** method processes controller and touch input, it updates the game state if any button was pressed.

C++

```

// Update the game state if the user chose a menu option.
if (m_startGameButton.IsPressed())
{
    SetGameState(GameState::PreGameCountdown);
    m_startGameButton.SetPressed(false);
}
if (m_highScoreButton.IsPressed())
{
    SetGameState(GameState::HighScoreDisplay);
    m_highScoreButton.SetPressed(false);
}

```

Controlling game play

The game loop and the **MarbleMazeMain::Update** method work together to update the state of game objects. If your game accepts input from multiple devices, you can accumulate the input from all devices into one set of variables so that you can write code that's easier to maintain. The **MarbleMazeMain::Update** method defines one set of variables that accumulates movement from all devices.

C++

```

float combinedTiltX = 0.0f;
float combinedTiltY = 0.0f;

```

The input mechanism can vary from one input device to another. For example, pointer input is handled by using the Windows Runtime event-handling model. Conversely, you poll for input data from the game controller when you need it. We recommend that you always follow the input mechanism that is prescribed for a given device. This section describes how Marble Maze reads input from each device, how it updates the combined input values, and how it uses the combined input values to update the state of the game.

Processing pointer input

When you work with pointer input, call the [Windows::UI::Core::CoreDispatcher::ProcessEvents](#) method to process window events. Call this method in your game loop before you update or render the scene. Marble Maze calls this in the **App::Run** method:

```
C++

while (!m_windowClosed)
{
    if (m_windowVisible)
    {
        CoreWindow::GetForCurrentThread()->
            Dispatcher-
                >ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);

        m_main->Update();

        if (m_main->Render())
        {
            m_deviceResources->Present();
        }
    }
    else
    {
        CoreWindow::GetForCurrentThread()->
            Dispatcher-
                >ProcessEvents(CoreProcessEventsOption::ProcessOneAndAllPending);
    }
}
```

If the window is visible, we pass **CoreProcessEventsOption::ProcessAllIfPresent** to **ProcessEvents** to process all queued events and return immediately; otherwise, we pass **CoreProcessEventsOption::ProcessOneAndAllPending** to process all queued events and wait for the next new event. After events are processed, Marble Maze renders and presents the next frame.

The Windows Runtime calls the registered handler for each event that occurred. The **App::SetWindow** method registers for events and forwards pointer information to the **MarbleMazeMain** class.

C++

```
void App::OnPointerPressed(
    Windows::UI::Core::CoreWindow^ sender,
    Windows::UI::Core::PointerEventArgs^ args)
{
    m_main->AddTouch(args->CurrentPoint->PointerId, args->CurrentPoint-
>Position);
}

void App::OnPointerReleased(
    Windows::UI::Core::CoreWindow^ sender,
    Windows::UI::Core::PointerEventArgs^ args)
{
    m_main->RemoveTouch(args->CurrentPoint->PointerId);
}

void App::OnPointerMoved(
    Windows::UI::Core::CoreWindow^ sender,
    Windows::UI::Core::PointerEventArgs^ args)
{
    m_main->UpdateTouch(args->CurrentPoint->PointerId, args->CurrentPoint-
>Position);
}
```

The **MarbleMazeMain** class reacts to pointer events by updating the map object that holds touch events. The **MarbleMazeMain::AddTouch** method is called when the pointer is first pressed, for example, when the user initially touches the screen on a touch-enabled device. The **MarbleMazeMain::UpdateTouch** method is called when the pointer position moves. The **MarbleMazeMain::RemoveTouch** method is called when the pointer is released, for example, when the user stops touching the screen.

C++

```
void MarbleMazeMain::AddTouch(int id, Windows::Foundation::Point point)
{
    m_touches[id] = PointToTouch(point, m_deviceResources-
>GetLogicalSize());

    m_pointQueue.push(D2D1::Point2F(point.X, point.Y));
}

void MarbleMazeMain::UpdateTouch(int id, Windows::Foundation::Point point)
{
    if (m_touches.find(id) != m_touches.end())
        m_touches[id] = PointToTouch(point, m_deviceResources-
```

```

>GetLogicalSize());
}

void MarbleMazeMain::RemoveTouch(int id)
{
    m_touches.erase(id);
}

```

The **PointToTouch** function translates the current pointer position so that the origin is in the center of the screen, and then scales the coordinates so that they range approximately between -1.0 and +1.0. This makes it easier to calculate the tilt of the maze in a consistent way across different input methods.

C++

```

inline XMFLOAT2 PointToTouch(Windows::Foundation::Point point,
Windows::Foundation::Size bounds)
{
    float touchRadius = min(bounds.Width, bounds.Height);
    float dx = (point.X - (bounds.Width / 2.0f)) / touchRadius;
    float dy = ((bounds.Height / 2.0f) - point.Y) / touchRadius;

    return XMFLOAT2(dx, dy);
}

```

The **MarbleMazeMain::Update** method updates the combined input values by incrementing the tilt factor by a constant scaling value. This scaling value was determined by experimenting with several different values.

C++

```

// Account for touch input.
for (TouchMap::const_iterator iter = m_touches.cbegin();
    iter != m_touches.cend();
    ++iter)
{
    combinedTiltX += iter->second.x * m_touchScaleFactor;
    combinedTiltY += iter->second.y * m_touchScaleFactor;
}

```

Processing accelerometer input

To process accelerometer input, the **MarbleMazeMain::Update** method calls the [Windows::Devices::Sensors::Accelerometer::GetCurrentReading](#) method. This method returns a [Windows::Devices::Sensors::AccelerometerReading](#) object, which represents an accelerometer reading. The

Windows::Devices::Sensors::AccelerometerReading::AccelerationX and **Windows::Devices::Sensors::AccelerometerReading::AccelerationY** properties hold the g-force acceleration along the X and Y axes, respectively.

The following example shows how the **MarbleMazeMain::Update** method polls the accelerometer and updates the combined input values. As you tilt the device, gravity causes the marble to move faster.

C++

```
// Account for sensors.
if (m_accelerometer != nullptr)
{
    Windows::Devices::Sensors::AccelerometerReading^ reading =
        m_accelerometer->GetCurrentReading();

    if (reading != nullptr)
    {
        combinedTiltX +=
            static_cast<float>(reading->AccelerationX) *
m_accelerometerScaleFactor;

        combinedTiltY +=
            static_cast<float>(reading->AccelerationY) *
m_accelerometerScaleFactor;
    }
}
```

Because you can't be sure that an accelerometer is present on the user's computer, always ensure that you have a valid [Accelerometer](#) object before you poll the accelerometer.

Processing game controller input

In the **MarbleMazeMain::Update** method, we use **m_newReading** to process input from the left analog stick:

C++

```
float leftStickX = static_cast<float>(m_newReading.LeftThumbstickX);
float leftStickY = static_cast<float>(m_newReading.LeftThumbstickY);

auto oppositeSquared = leftStickY * leftStickY;
auto adjacentSquared = leftStickX * leftStickX;

if ((oppositeSquared + adjacentSquared) > m_deadzoneSquared)
{
    combinedTiltX += leftStickX * m_controllerScaleFactor;
```

```
combinedTiltY += leftStickY * m_controllerScaleFactor;  
}
```

We check if the input from the left analog stick is outside of the dead zone, and if it is, we add it to **combinedTiltX** and **combinedTiltY** (multiplied by a scale factor) to tilt the stage.

Important

When you work with a game controller, always account for the dead zone. The dead zone refers to the variance among gamepads in their sensitivity to initial movement. In some controllers, a small movement may generate no reading, but in others it may generate a measurable reading. To account for this in your game, create a zone of non-movement for initial thumbstick movement. For more information about the dead zone, see [Reading the thumbsticks](#).

Applying input to the game state

Devices report input values in different ways. For example, pointer input might be in screen coordinates, and controller input might be in a completely different format. One challenge with combining input from multiple devices into one set of input values is normalization, or converting values to a common format. Marble Maze normalizes values by scaling them to the range [-1.0, 1.0]. The **PointToTouch** function, which is previously described in this section, converts screen coordinates to normalized values that range approximately between -1.0 and +1.0.

Tip

Even if your application uses one input method, we recommend that you always normalize input values. Doing so can simplify how input is interpreted by other components of your game, such as physics simulation, and makes it easier to write games that work on different screen resolutions.

After the **MarbleMazeMain::Update** method processes input, it creates a vector that represents the effect of the tilt of the maze on the marble. The following example shows how Marble Maze uses the [XMVector3Normalize](#) function to create a normalized gravity

vector. The **maxTilt** variable constrains the amount by which the maze tilts and prevents the maze from tilting on its side.

C++

```
const float maxTilt = 1.0f / 8.0f;

XMVECTOR gravity = XMVectorSet(
    combinedTiltX * maxTilt,
    combinedTiltY * maxTilt,
    1.0f,
    0.0f);

gravity = XMVector3Normalize(gravity);
```

To complete the update of scene objects, Marble Maze passes the updated gravity vector to the physics simulation, updates the physics simulation for the time that has elapsed since the previous frame, and updates the position and orientation of the marble. If the marble has fallen through the maze, the **MarbleMazeMain::Update** method places the marble back at the last checkpoint that the marble touched and resets the state of the physics simulation.

C++

```
XMFLOAT3A g;
XMStoreFloat3(&g, gravity);
m_physics.SetGravity(g);

if (m_gameState == GameState::InGameActive)
{
    // Only update physics when gameplay is active.
    m_physics.UpdatePhysicsSimulation(static_cast<float>
(m_timer.GetElapsedSeconds()));

    // ...Code omitted for simplicity...
}

// ...Code omitted for simplicity...

// Check whether the marble fell off of the maze.
const float fadeOutDepth = 0.0f;
const float resetDepth = 80.0f;
if (marblePosition.z >= fadeOutDepth)
{
    m_targetLightStrength = 0.0f;
}
if (marblePosition.z >= resetDepth)
{
    // Reset marble.
```

```
    memcpy(&marblePosition, &m_checkpoints[m_currentCheckpoint],
sizeof(XMFLOAT3));
    oldMarblePosition = marblePosition;
    m_physics.SetPosition((const XMFLOAT3&)marblePosition);
    m_physics.SetVelocity(XMFLOAT3(0, 0, 0));
    m_lightStrength = 0.0f;
    m_targetLightStrength = 1.0f;

    m_resetCamera = true;
    m_resetMarbleRotation = true;
    m_audio.PlaySoundEffect(FallingEvent);
}
```

This section does not describe how the physics simulation works. For details about that, see [Physics.h](#) and [Physics.cpp](#) in the Marble Maze sources.

Next steps

Read [Adding audio to the Marble Maze sample](#) for information about some of the key practices to keep in mind when you work with audio. The document discusses how Marble Maze uses Microsoft Media Foundation and XAudio2 to load, mix, and play audio resources.

Related topics

- [Adding audio to the Marble Maze sample](#)
- [Adding visual content to the Marble Maze sample](#)
- [Developing Marble Maze, a UWP game in C++ and DirectX](#)

Adding audio to the Marble Maze sample

Article • 10/20/2022

This document describes the key practices to consider when you work with audio and shows how Marble Maze applies these practices. Marble Maze uses [Microsoft Media Foundation](#) to load audio resources from files, and [XAudio2](#) to mix and play audio and to apply effects to audio.

Marble Maze plays music in the background, and also uses gameplay sounds to indicate game events, such as when the marble hits a wall. An important part of the implementation is that Marble Maze uses a reverb, or echo, effect to simulate the sound of a marble when it bounces. The reverb effect implementation causes echoes to reach you more quickly and loudly in small rooms; echoes are quieter and reach you more slowly in larger rooms.

ⓘ Note

The sample code that corresponds to this document is found in the [DirectX Marble Maze game sample](#) [↗](#).

Here are some of the key points that this document discusses for when you work with audio in your game:

- Consider using Media Foundation to decode audio assets and XAudio2 to play audio. However, if you have an existing audio asset-loading mechanism that works in a Universal Windows Platform (UWP) app, you can use it.
- An audio graph contains one source voice for each active sound, zero or more submix voices, and one mastering voice. Source voices can feed into submix voices and/or the mastering voice. Submix voices feed into other submix voices or the mastering voice.
- If your background music files are large, consider streaming your music into smaller buffers so that less memory is used.
- If it makes sense to do so, pause audio playback when the app loses focus or visibility, or is suspended. Resume playback when your app regains focus, becomes visible, or is resumed.

- Set audio categories to reflect the role of each sound. For example, you typically use **AudioCategory_GameMedia** for game background audio and **AudioCategory_GameEffects** for sound effects.
- Handle device changes, including headphones, by releasing and recreating all audio resources and interfaces.
- Consider whether to compress audio files when minimizing disk space and streaming costs is a requirement. Otherwise, you can leave audio uncompressed so that it loads faster.

Introducing XAudio2 and Microsoft Media Foundation

XAudio2 is a low-level audio library for Windows that specifically supports game audio. It provides a digital signal processing (DSP) and audio-graph engine for games. XAudio2 expands on its predecessors, DirectSound and XAudio, by supporting computing trends such as SIMD floating-point architectures and HD audio. It also supports the more complex sound processing demands of today's games.

The document [XAudio2 Key Concepts](#) explains the key concepts for using XAudio2. In brief, the concepts are:

- The [IXAudio2](#) interface is the core of the XAudio2 engine. Marble Maze uses this interface to create voices and to receive notification when the output device changes or fails.
- A **voice** processes, adjusts, and plays audio data.
- A **source voice** is a collection of audio channels (mono, 5.1, and so on) and represents one stream of audio data. In XAudio2, a source voice is where audio processing begins. Typically, sound data is loaded from an external source, such as a file or a network, and is sent to a source voice. Marble Maze uses [Media Foundation](#) to load sound data from files. Media Foundation is introduced later in this document.
- A **submix voice** processes audio data. This processing can include changing the audio stream or combining multiple streams into one. Marble Maze uses submixes to create the reverb effect.
- A **mastering voice** combines data from source and submix voices and sends that data to the audio hardware.

- An **audio graph** contains one source voice for each active sound, zero or more submix voices, and only one mastering voice.
- A **callback** informs client code that some event has occurred in a voice or in an engine object. By using callbacks, you can reuse memory when XAudio2 is finished with a buffer, react when the audio device changes (for example, when you connect or disconnect headphones), and more. [Handling headphones and device changes](#) later in this document explains how Marble Maze uses this mechanism to handle device changes.

Marble Maze uses two audio engines (in other words, two [IXAudio2](#) objects) to process audio. One engine processes the background music, and the other engine processes gameplay sounds.

Marble Maze must also create one mastering voice for each engine. Recall that a mastering engine combines audio streams into one stream and sends that stream to the audio hardware. The background music stream, a source voice, outputs data to a mastering voice and to two submix voices. The submix voices perform the reverb effect.

Media Foundation is a multimedia library that supports many audio and video formats. XAudio2 and Media Foundation complement each other. Marble Maze uses Media Foundation to load audio assets from files and uses XAudio2 to play audio. You don't have to use Media Foundation to load audio assets. If you have an existing audio asset loading mechanism that works in Universal Windows Platform (UWP) apps, use it. [Audio, video, and camera](#) discusses several ways of implementing audio in a UWP app.

For more information about XAudio2, see [Programming Guide](#). For more information about Media Foundation, see [Microsoft Media Foundation](#).

Initializing audio resources

Marble Mazes uses a Windows Media Audio (.wma) file for the background music, and WAV (.wav) files for gameplay sounds. These formats are supported by Media Foundation. Although the .wav file format is natively supported by XAudio2, a game has to parse the file format manually to fill out the appropriate XAudio2 data structures. Marble Maze uses Media Foundation to more easily work with .wav files. For the complete list of the media formats that are supported by Media Foundation, see [Supported Media Formats in Media Foundation](#). Marble Maze does not use separate design-time and run-time audio formats, and does not use XAudio2 ADPCM compression support. For more information about ADPCM compression in XAudio2, see [ADPCM Overview](#).

The `Audio::CreateResources` method, which is called from `MarbleMazeMain::LoadDeferredResources`, loads the audio streams from the files, initializes the XAudio2 engine objects, and creates the source, submix, and mastering voices.

Creating the XAudio2 engines

Recall that Marble Maze creates one `IXAudio2` object to represent each audio engine that it uses. To create an audio engine, call the `XAudio2Create` method. The following example shows how Marble Maze creates the audio engine that processes background music.

C++

```
// In Audio.h
class Audio
{
private:
    IXAudio2*                m_musicEngine;
// ...
}

// In Audio.cpp
void Audio::CreateResources()
{
    try
    {
        // ...
        DX::ThrowIfFailed(
            XAudio2Create(&m_musicEngine)
        );
        // ...
    }
    // ...
}
```

Marble Maze performs a similar step to create the audio engine that plays gameplay sounds.

How to work with the `IXAudio2` interface in a UWP app differs from a desktop app in two ways. First, you don't have to call `ColInitializeEx` before you call `XAudio2Create`. In addition, `IXAudio2` no longer supports device enumeration. For information about how to enumerate audio devices, see [Enumerating devices](#).

Creating the mastering voices

The following example shows how the **Audio::CreateResources** method creates the mastering voice for the background music using the [IXAudio2::CreateMasteringVoice](#) method. In this example, **m_musicMasteringVoice** is an [IXAudio2MasteringVoice](#) object. We specify two input channels; this simplifies the logic for the reverb effect.

We specify 48000 as the input sample rate. We chose this sample rate because it represented a balance between audio quality and the amount of required CPU processing. A greater sample rate would have required more CPU processing without having a noticeable quality benefit.

Finally, we specify **AudioCategory_GameMedia** as the audio stream category so that users can listen to music from a different application as they play the game. When a music app is playing, Windows mutes any voices that are created by the **AudioCategory_GameMedia** option. The user still hears gameplay sounds because they are created by the **AudioCategory_GameEffects** option. For more info about audio categories, see [AUDIO_STREAM_CATEGORY](#).

C++

```
// This sample plays the equivalent of background music, which we tag on the
// mastering voice as AudioCategory_GameMedia. In ordinary usage, if we were
// playing the music track with no effects, we could route it entirely
// through
// Media Foundation. Here, we are using XAudio2 to apply a reverb effect to
// the
// music, so we use Media Foundation to decode the data then we feed it
// through
// the XAudio2 pipeline as a separate Mastering Voice, so that we can tag it
// as Game Media. We default the mastering voice to 2 channels to simplify
// the reverb logic.
DX::ThrowIfFailed(
    m_musicEngine->CreateMasteringVoice(
        &m_musicMasteringVoice,
        2,
        48000,
        0,
        nullptr,
        nullptr,
        AudioCategory_GameMedia
    )
);
```

The **Audio::CreateResources** method performs a similar step to create the mastering voice for the gameplay sounds, except that it specifies **AudioCategory_GameEffects** for the *StreamCategory* parameter, which is the default.

Creating the reverb effect

For each voice, you can use XAudio2 to create sequences of effects that process audio. Such a sequence is known as an effect chain. Use effect chains when you want to apply one or more effects to a voice. Effect chains can be destructive; that is, each effect in the chain can overwrite the audio buffer. This property is important because XAudio2 makes no guarantee that output buffers are initialized with silence. Effect objects are represented in XAudio2 by cross-platform audio processing objects (XAPO). For more information about XAPO, see [XAPO Overview](#).

When you create an effect chain, follow these steps:

1. Create the effect object.
2. Populate an [XAUDIO2_EFFECT_DESCRIPTOR](#) structure with effect data.
3. Populate an [XAUDIO2_EFFECT_CHAIN](#) structure with data.
4. Apply the effect chain to a voice.
5. Populate an effect parameter structure and apply it to the effect.
6. Disable or enable the effect whenever appropriate.

The **Audio** class defines the **CreateReverb** method to create the effect chain that implements reverb. This method calls the [XAudio2CreateReverb](#) method to create a **ComPtr<IUnknown>** object, **soundEffectXAPO**, which acts as the submix voice for the reverb effect.

C++

```
Microsoft::WRL::ComPtr<IUnknown> soundEffectXAPO;  
  
DX::ThrowIfFailed(  
    XAudio2CreateReverb(&soundEffectXAPO)  
);
```

The [XAUDIO2_EFFECT_DESCRIPTOR](#) structure contains information about an XAPO for use in an effect chain, for example, the target number of output channels. The **Audio::CreateReverb** method creates an [XAUDIO2_EFFECT_DESCRIPTOR](#) object, **soundEffectdescriptor**, that is set to the disabled state, uses two output channels, and references **soundEffectXAPO** for the reverb effect. **soundEffectdescriptor** starts in the disabled state because the game must set parameters before the effect starts modifying game sounds. Marble Maze uses two output channels to simplify the logic for the reverb effect.

C++

```
soundEffectDescriptor.InitialState = false;
soundEffectDescriptor.OutputChannels = 2;
soundEffectDescriptor.pEffect = soundEffectXAPO.Get();
```

If your effect chain has multiple effects, each effect requires an object. The [XAUDIO2_EFFECT_CHAIN](#) structure holds the array of [XAUDIO2_EFFECT_DESCRIPTOR](#) objects that participate in the effect. The following example shows how the **Audio::CreateReverb** method specifies the one effect to implement reverb.

```
C++

XAUDIO2_EFFECT_CHAIN soundEffectChain;

// ...

soundEffectChain.EffectCount = 1;
soundEffectChain.pEffectDescriptors = &soundEffectDescriptor;
```

The **Audio::CreateReverb** method calls the [IXAudio2::CreateSubmixVoice](#) method to create the submix voice for the effect. It specifies the [XAUDIO2_EFFECT_CHAIN](#) object, **soundEffectChain**, for the *pEffectChain* parameter to associate the effect chain with the voice. Marble Maze also specifies two output channels and a sample rate of 48 kilohertz.

```
C++

DX::ThrowIfFailed(
    engine->CreateSubmixVoice(newSubmix, 2, 48000, 0, 0, nullptr,
    &soundEffectChain)
);
```

Tip

If you want to attach an existing effect chain to an existing submix voice, or you want to replace the current effect chain, use the [IXAudio2Voice::SetEffectChain](#) method.

The **Audio::CreateReverb** method calls [IXAudio2Voice::SetEffectParameters](#) to set additional parameters that are associated with the effect. This method takes a parameter structure that is specific to the effect. An [XAUDIO2FX_REVERB_PARAMETERS](#) object, **m_reverbParametersSmall**, which contains the effect parameters for reverb, is initialized in the **Audio::Initialize** method because every reverb effect shares the same parameters. The following example shows how the **Audio::Initialize** method initializes the reverb parameters for near-field reverb.

C++

```
m_reverbParametersSmall.ReflectionsDelay =
XAUDIO2FX_REVERB_DEFAULT_REFLECTIONS_DELAY;
m_reverbParametersSmall.ReverbDelay = XAUDIO2FX_REVERB_DEFAULT_REVERB_DELAY;
m_reverbParametersSmall.RearDelay = XAUDIO2FX_REVERB_DEFAULT_REAR_DELAY;
m_reverbParametersSmall.PositionLeft = XAUDIO2FX_REVERB_DEFAULT_POSITION;
m_reverbParametersSmall.PositionRight = XAUDIO2FX_REVERB_DEFAULT_POSITION;
m_reverbParametersSmall.PositionMatrixLeft =
XAUDIO2FX_REVERB_DEFAULT_POSITION_MATRIX;
m_reverbParametersSmall.PositionMatrixRight =
XAUDIO2FX_REVERB_DEFAULT_POSITION_MATRIX;
m_reverbParametersSmall.EarlyDiffusion = 4;
m_reverbParametersSmall.LateDiffusion = 15;
m_reverbParametersSmall.LowEQGain = XAUDIO2FX_REVERB_DEFAULT_LOW_EQ_GAIN;
m_reverbParametersSmall.LowEQCutoff =
XAUDIO2FX_REVERB_DEFAULT_LOW_EQ_CUTOFF;
m_reverbParametersSmall.HighEQGain = XAUDIO2FX_REVERB_DEFAULT_HIGH_EQ_GAIN;
m_reverbParametersSmall.HighEQCutoff =
XAUDIO2FX_REVERB_DEFAULT_HIGH_EQ_CUTOFF;
m_reverbParametersSmall.RoomFilterFreq =
XAUDIO2FX_REVERB_DEFAULT_ROOM_FILTER_FREQ;
m_reverbParametersSmall.RoomFilterMain =
XAUDIO2FX_REVERB_DEFAULT_ROOM_FILTER_MAIN;
m_reverbParametersSmall.RoomFilterHF =
XAUDIO2FX_REVERB_DEFAULT_ROOM_FILTER_HF;
m_reverbParametersSmall.ReflectionsGain =
XAUDIO2FX_REVERB_DEFAULT_REFLECTIONS_GAIN;
m_reverbParametersSmall.ReverbGain = XAUDIO2FX_REVERB_DEFAULT_REVERB_GAIN;
m_reverbParametersSmall.DecayTime = XAUDIO2FX_REVERB_DEFAULT_DECAY_TIME;
m_reverbParametersSmall.Density = XAUDIO2FX_REVERB_DEFAULT_DENSITY;
m_reverbParametersSmall.RoomSize = XAUDIO2FX_REVERB_DEFAULT_ROOM_SIZE;
m_reverbParametersSmall.WetDryMix = XAUDIO2FX_REVERB_DEFAULT_WET_DRY_MIX;
m_reverbParametersSmall.DisableLateField = TRUE;
```

This example uses the default values for most of the reverb parameters, but it sets **DisableLateField** to TRUE to specify near-field reverb, **EarlyDiffusion** to 4 to simulate flat near surfaces, and **LateDiffusion** to 15 to simulate very diffuse distant surfaces. Flat near surfaces cause echoes to reach you more quickly and loudly; diffuse distant surfaces cause echoes to be quieter and reach you more slowly. You can experiment with reverb values to get the desired effect in your game or use the **ReverbConvertI3DL2ToNative** method to use industry-standard I3DL2 (Interactive 3D Audio Rendering Guidelines Level 2.0) parameters.

The following example shows how **Audio::CreateReverb** sets the reverb parameters. **newSubmix** is an **IXAudio2SubmixVoice**** object. **parameters** is an **XAUDIO2FX_REVERB_PARAMETERS*** object.

C++

```

DX::ThrowIfFailed(
    (*newSubmix)->SetEffectParameters(0, parameters,
    sizeof(m_reverbParametersSmall))
);

```

The **Audio::CreateReverb** method finishes by enabling the effect using [IXAudio2Voice::EnableEffect](#) if the **enableEffect** flag is set. It also sets its volume using [IXAudio2Voice::SetVolume](#) and output matrix using [IXAudio2Voice::SetOutputMatrix](#). This part sets the volume to full (1.0) and then specifies the volume matrix to be silence for both left and right inputs and left and right output speakers. We do this because other code later cross-fades between the two reverbs (simulating the transition from being near a wall to being in a large room), or mutes both reverbs if required. When the reverb path is later unmuted, the game sets a matrix of {1.0f, 0.0f, 0.0f, 1.0f} to route left reverb output to the left input of the mastering voice and right reverb output to the right input of the mastering voice.

C++

```

if (enableEffect)
{
    DX::ThrowIfFailed(
        (*newSubmix)->EnableEffect(0)
    );
}

DX::ThrowIfFailed(
    (*newSubmix)->SetVolume (1.0f)
);

float outputMatrix[4] = {0, 0, 0, 0};
DX::ThrowIfFailed(
    (*newSubmix)->SetOutputMatrix(masteringVoice, 2, 2, outputMatrix)
);

```

Marble Maze calls the **Audio::CreateReverb** method four times: two times for the background music and two times for the gameplay sounds. The following shows how Marble Maze calls the **CreateReverb** method for the background music.

C++

```

CreateReverb(
    m_musicEngine,
    m_musicMasteringVoice,
    &m_reverbParametersSmall,
    &m_musicReverbVoiceSmallRoom,
    true
);

```

```
CreateReverb(
    m_musicEngine,
    m_musicMasteringVoice,
    &m_reverbParametersLarge,
    &m_musicReverbVoiceLargeRoom,
    true
);
```

For a list of possible sources of effects for use with XAudio2, see [XAudio2 Audio Effects](#).

Loading audio data from file

Marble Maze defines the **MediaStreamer** class, which uses Media Foundation to load audio resources from files. Marble Maze uses one **MediaStreamer** object to load each audio file.

Marble Maze calls the **MediaStreamer::Initialize** method to initialize each audio stream. Here's how the **Audio::CreateResources** method calls **MediaStreamer::Initialize** to initialize the audio stream for the background music:

C++

```
// Media Foundation is a convenient way to get both file I/O and format
// decode for
// audio assets. You can replace the streamer in this sample with your own
// file I/O
// and decode routines.
m_musicStreamer.Initialize(L"Media\\Audio\\background.wma");
```

The **MediaStreamer::Initialize** method starts by calling the [MFStartup](#) method to initialize Media Foundation. **MF_VERSION** is a macro defined in **mfapi.h**, and is what we specify as the version of Media Foundation to use.

C++

```
DX::ThrowIfFailed(
    MFStartup(MF_VERSION)
);
```

MediaStreamer::Initialize then calls [MFCreateSourceReaderFromURL](#) to create an [IMFSourceReader](#) object. An **IMFSourceReader** object, **m_reader**, reads media data from the file that is specified by **url**.

C++